

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gabriela Pricop, FAF-241

Report

Laboratory Work No. 12

Operating Systems

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

Goal **3**

1 Part 1: Producer and consumer **3**

2 Part 2: The condition variable **7**

3 Part 3: Readers and writers **10**

4 Part 4: Build a deadlock, then remove it **13**

Conclusion **18**

References **18**

Goal

In the previous laboratory work one semaphore was used at a time, in its three roles: as a lock, as a signal and as a counter. The purpose of this laboratory work is to make the tools work together, because a real problem almost never needs only one of them. Two semaphores and a mutex are combined to solve the two classical problems of every operating systems course, the producer and the consumer and the readers and the writers; a new tool, the condition variable, is met; and a deadlock is built on purpose and then removed.

The work has four parts. The first part is the producer and the consumer around a buffer of four slots, where one semaphore counts the free slots, another counts the filled ones and a mutex protects the buffer itself; this is exactly how a pipe of Unix works inside the kernel. The second part solves the problem of the order from the previous laboratory work in another way: the state is kept in an ordinary variable protected by a mutex, and a condition variable is only the place where a thread sleeps until another one announces that something has changed. The third part allows many readers to read at the same time while a writer must be alone. The fourth part builds a deadlock from two mutexes taken in opposite order and then removes it by imposing one single order.

Part 1: Producer and consumer

Questions:

- The producer is fast, yet "in buffer" never goes above 4. Which call stops it?
- Who waits at the start: the consumer or the producer? Which call makes it wait?
- In producer(), swap the first two lines: lock the mutex before sem_wait(&empty). Run it. What happens when the buffer is full, and why?

Answers:

- The call sem_wait(&empty), which is the first line of the loop of the producer. The semaphore empty starts at SIZE, that is 4, and counts the free slots: every put lowers it by one and every take raises it by one through sem_post(&empty). When the four slots are filled the value is 0, so the fifth sem_wait finds nothing to take and the producer is put to sleep until the consumer frees a slot. The buffer is an array of exactly four integers, so this is not a convenience but a necessity: without that call the producer would write past the end of the array and over items that had not been taken yet.
- The consumer waits, at the call sem_wait(&full). The semaphore full starts at 0, because at the beginning nothing has been produced, so the consumer blocks immediately; it is also created first in main, on purpose, so that it certainly arrives before the producer. It is woken by the sem_post(&full) that the producer performs after placing the first item. The two semaphores are therefore mirror images of each other, as shown in Table 1.
- The program would stop for ever as soon as the buffer became full, which is a dead-

lock. The producer would hold the mutex and then block inside sem_wait(&empty), waiting for a free slot; the only thread that can free a slot is the consumer, but to take an item it must first pass pthread_mutex_lock(&lock), and the mutex is held by the producer, which is asleep. Each of the two waits for something that only the other can give, and neither can give it, so nothing moves and the program has to be stopped with Ctrl-C. The rule that this illustrates is that one must never fall asleep while holding a lock: the semaphore that may block must be taken before the mutex, never inside it. This is the same circle that is built deliberately in the fourth part.

```
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab12$ ./prodcons
put 1 (in buffer: 1)
put 2 (in buffer: 2)
 take 1 (in buffer: 1)
put 3 (in buffer: 2)
put 4 (in buffer: 3)
put 5 (in buffer: 4)
```

Figure 1.1: A second run: the buffer became full

prodcons.c – three tools work together. The semaphore empty counts the free slots and starts at 4, the semaphore full counts the filled slots and starts at 0, and the mutex protects the buffer and the indexes while they are being changed. The buffer is circular: the indexes in and out advance with the remainder of the division by SIZE, so they return to the beginning after the fourth slot. The consumer is made slow on purpose with usleep(300000), that is three tenths of a second after every item, so that the producer really does reach the full buffer and the limit becomes visible. The source is given below.

```c
/* prodcons.c - producer and consumer around a buffer of 4 slots
   .
 *   empty = free slots (starts at SIZE), full = filled slots (
   starts at 0),
 *   lock = the buffer itself.
 * Build: gcc -Wall -pthread -o prodcons prodcons.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE  4
#define ITEMS 8

int buf[SIZE];
int in = 0, out = 0, count = 0;       /* count only for printing
   */
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);             /* wait for a free slot */
```

```
22          pthread_mutex_lock(&lock);
23          buf[in] = item;
24          in = (in + 1) % SIZE;
25          count++;
26          printf("put %d    (in buffer: %d)\n", item, count);
27          pthread_mutex_unlock(&lock);
28          sem_post(&full);              /* one more filled slot */
29      }
30      return NULL;
31  }
32
33  void *consumer(void *arg) {
34      for (int k = 0; k < ITEMS; k++) {
35          sem_wait(&full);              /* wait for something to
        take */
36          pthread_mutex_lock(&lock);
37          int item = buf[out];
38          out = (out + 1) % SIZE;
39          count--;
40          printf("            take %d   (in buffer: %d)\n", item,
        count);
41          pthread_mutex_unlock(&lock);
42          sem_post(&empty);             /* one more free slot */
43          usleep(300000);               /* the consumer is slow */
44      }
45      return NULL;
46  }
47
48  int main(void) {
49      sem_init(&empty, 0, SIZE);
50      sem_init(&full, 0, 0);
51      pthread_t p, c;
52      pthread_create(&c, NULL, consumer, NULL);
53      pthread_create(&p, NULL, producer, NULL);
54      pthread_join(p, NULL);
55      pthread_join(c, NULL);
56      sem_destroy(&empty);
57      sem_destroy(&full);
58      return 0;
59  }
```

```
 GNU nano 8.7.1                    prodcons.c *
/* prodcons.c - producer and consumer around a buffer of 4 slots.
* empty = free slots (starts at SIZE), full = filled slots (starts at 0), lock >
buffer itself.
* Build: gcc -Wall -pthread -o prodcons prodcons.c
*/
#include <stdio.h>
#include <unistd.h>
```

Figure 1.2: The beginning of prodcons.c in the editor

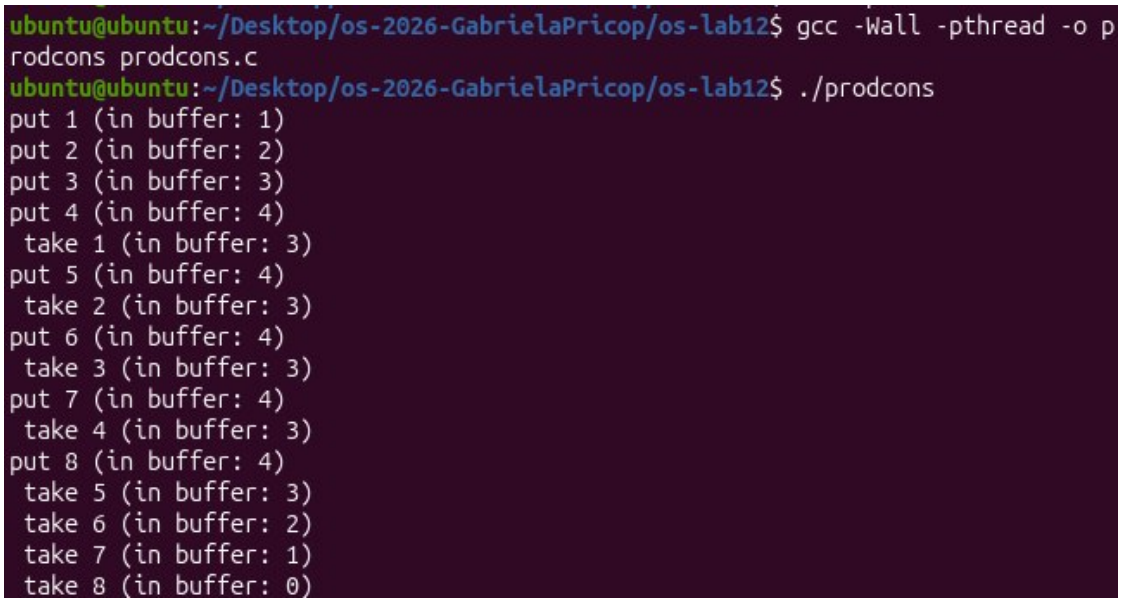


Figure 1.3: A complete run: the buffer fills to four and never further

gcc -Wall -pthread -o prodcons prodcons.c ; ./prodcons – the run shows the mechanism exactly. The first four items are put one after the other, without any pause, and the count rises 1, 2, 3, 4: the producer is fast and the semaphore empty had four units to give. Then the producer stops, because empty has reached 0, and nothing more is printed until the consumer takes the first item; the count falls to 3, a slot is freed, and the producer immediately puts the item 5 and brings the count back to 4. From there the two threads advance in lockstep, one take followed by one put, each time between 3 and 4, until the producer has placed all eight items; the consumer then empties the buffer alone, 3, 2, 1, 0, with its three tenths of a second between the lines. The number in the buffer is never 5.

Tool	**Starts at**	**Who waits on it**	**What it guarantees**
empty	4	the producer, before putting	never more than 4 items in the buffer
full	0	the consumer, before taking	nothing is taken from an empty slot
lock	free	both, around the buffer	the indexes and the count stay consistent

Table 1: The three tools of the producer and the consumer

Table 1 shows that the two semaphores do not protect anything: they count, and the counting is what blocks a thread at the right moment. The mutex protects, but it cannot count. Neither of the two could solve the problem alone, which is the lesson of this part.

Part 2: The condition variable

Questions:

- In order.c of the previous laboratory work the semaphore itself remembered that the message was ready. Here, which variable remembers it?
- Replace while (!ready) with if (!ready), compile, run. Does it still work here?
- Then why do we write while? Give one reason why a thread can wake up and still find ready == 0.

Answers:

- The ordinary integer ready, protected by the mutex. This is the essential difference between the two solutions: a semaphore is itself a piece of memory, and its value is the state, so posting it both records the event and wakes the waiter; a condition variable remembers nothing at all and keeps no value. It is only a waiting room, and the state must be kept beside it, in a normal variable that the programmer declares and that the mutex protects. If the writer set ready to 1 and forgot to signal, the reader would sleep for ever; if it signalled without setting ready, the reader would wake up, find the condition false and sleep again.
- Yes, in this program it still works, and the output is exactly the same: B: not ready, I sleep, then A: message written, then B: I read "hello". The reason is that this program is the easiest possible case: there is a single waiting thread, a single signal, and the only thread that can change the condition is the writer, so when the reader wakes up the condition is certainly true. The experiment is therefore misleading, and that is its real lesson: a test that passes does not prove that the code is correct, it only proves that this run did not meet the case that breaks it.

```
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab12$ nano condvar.c
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab12$ gcc -Wall -pthread -o c
ondvar condvar.c
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab12$ ./condvar
B: not ready, I sleep
A: message written
B: I read "hello"
```

Figure 2.1: The run with the if statement

- Because a thread can return from pthread_cond_wait without the condition being true. There are three reasons, and one is enough to answer. First, the call is allowed to return spuriously, that is without anybody having signalled, and the standard explicitly permits it because it makes the implementation faster on several systems. Second, pthread_cond_wait must take the mutex back before it returns, so between the moment the thread is woken and the moment it actually runs, another thread may enter the critical section first and consume the condition; the woken thread then finds nothing left. Third, pthread_cond_broadcast wakes every waiter, although often only one of them can really proceed. In all three cases the only safe behaviour is to test the condition again after waking, which is exactly what the while loop does and what an if cannot do.

condvar.c – the writer takes the mutex, copies the message, sets ready to 1, prints its line and calls pthread_cond_signal while still holding the mutex, which it releases immediately afterwards. The reader takes the mutex and tests the condition inside a while loop; if the message is not ready, it calls pthread_cond_wait, which performs three actions in one indivisible step: it releases the mutex, puts the thread to sleep and, when it is woken, takes the mutex again before returning. That the release and the sleep are indivisible is what makes the tool safe: if the reader released the mutex and then went to sleep in two separate steps, the writer could signal exactly in between, and the signal would be lost because nobody would be waiting yet. The source is given below.

```c
/* condvar.c - the order.c problem, solved with a mutex and a condition variable.
 * Build: gcc -Wall -pthread -o condvar condvar.c
 */
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>

char message[32];
int ready = 0;                          /* the condition itself */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t changed = PTHREAD_COND_INITIALIZER;

void *writer(void *arg) {
    sleep(1);
    pthread_mutex_lock(&lock);
    strcpy(message, "hello");
    ready = 1;
    printf("A: message written\n");
    pthread_cond_signal(&changed);  /* wake one waiter */
    pthread_mutex_unlock(&lock);
    return NULL;
}

void *reader(void *arg) {
    pthread_mutex_lock(&lock);
    while (!ready) {                /* while, not if: re-check after waking */
        printf("B: not ready, I sleep\n");
        pthread_cond_wait(&changed, &lock);  /* unlocks, sleeps, locks again */
    }
    printf("B: I read \"%s\"\n", message);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);
```

```
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
```

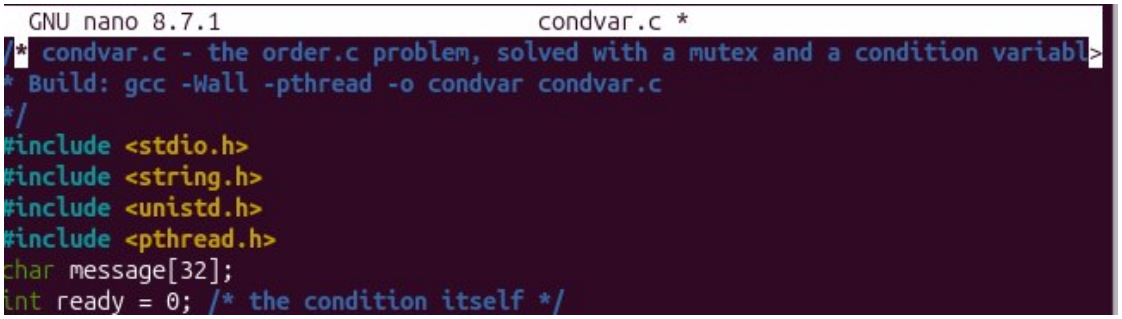


Figure 2.2: The beginning of condvar.c in the editor

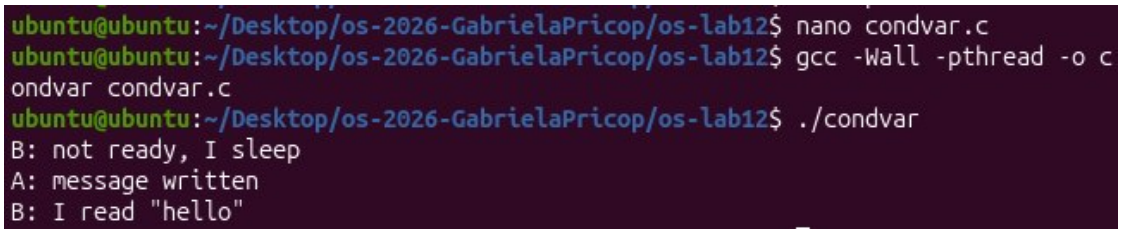


Figure 2.3: The run with the while loop: the three lines in the right order

gcc -Wall -pthread -o condvar condvar.c ; ./condvar – the output is the expected one: B: not ready, I sleep appears at once, then nothing happens for one second, then A: message written and immediately after it B: I read "hello". The pause is the proof that the reader was really sleeping and not testing the variable in a loop, and the order of the last two lines is the proof that the reader could not run before the writer released the mutex.

./condvar (with if instead of while) – the program was edited, compiled and run again for the second question, and it printed exactly the same three lines. The condition was true when the reader woke up, so the difference between the two versions never showed itself. The version that is kept in the repository is the one with while, for the reasons given in the third answer. The comparison between the two solutions of the same problem is given in Table 2.

Aspect	**order.c: semaphore at 0**	**condvar.c: condition variable**
where the state is	inside the semaphore, as its value	in the variable ready, beside it
what the waiter does	sem_wait, once	tests the condition in a while loop
if the signal comes first	it is remembered, the waiter does not block	it is lost, but ready is 1 so nobody waits
needs a mutex	no	yes, the condition must be protected

Table 2: The same problem of order, solved in two ways

Table 2 shows the practical difference between the two tools. A semaphore counts and therefore remembers, so an early post is not lost; a condition variable does not remember anything, and that is precisely why the condition must be a separate variable and why it must be tested again after every waking.

Part 3: Readers and writers

Questions:

- Three readers are inside at the same time, but a writer is always alone. Which two lines of reader() make that possible?
- Why does the counter readers need its own lock (rlock)? What could go wrong without it?
- Imagine readers that keep arriving without a pause. What happens to writer 2?

Answers:

- The two lines that test the counter: if (++readers == 1) sem_wait(&room); at the entrance and if (–readers == 0) sem_post(&room); at the exit. Only the first reader to arrive takes the semaphore room, and only the last one to leave gives it back; every reader in between simply increments the counter and walks in, without touching room at all. The room is therefore held by the group of readers and not by any particular one of them, which is why three readers can be inside together while a writer, which always takes room for itself alone, has to wait for the whole group to leave.
- Because readers is an ordinary shared integer, and ++readers and –readers are the read, add, write of the first part of the previous laboratory work. Without rlock two readers could both read the value 0, both compute 1, and both believe they are the first: both would then call sem_wait(&room), the second one would block for ever, because room has only one unit, although it should have entered freely. Symmetrically, two readers leaving at the same time could both see the counter reach 0 and both call sem_post(&room), which would raise the semaphore to 2 and let two writers write at the same time, destroying the whole guarantee. The lock

makes the increment and the test of its result one single indivisible operation, which is exactly what is needed.

- Writer 2 starves: it waits for ever, although nothing is broken and no deadlock exists. The semaphore room is released only when the counter falls to zero, that is when the last reader leaves; if new readers keep arriving before that happens, the counter never reaches zero, room is never posted, and the writer stays in the queue indefinitely. This solution gives priority to the readers, and the problem has a name, starvation: a thread is never blocked by a circle, it is simply always overtaken. The cure is to make the arriving readers wait when a writer is already waiting, which is the writer-preference variant of the same classical problem.

readers.c – two semaphores, both started at 1 but with different jobs. The semaphore room means that the data is free, and a writer takes it for the whole duration of its writing; the semaphore rlock is an ordinary lock, used only around the counter readers and held for a few instructions. The source is given below.

```c
/* readers.c - readers and writers with two semaphores.
 *   Many readers may read together; a writer must be alone.
 * Build: gcc -Wall -pthread -o readers readers.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define READERS 3
#define WRITERS 2

int value = 0;          /* the shared data */
int readers = 0;        /* how many are reading now */
sem_t rlock;            /* protects the counter readers */
sem_t room;             /* 1 = the room is free for a writer */

void *reader(void *arg) {
    long id = (long)arg;
    sem_wait(&rlock);
    if (++readers == 1)       /* the first reader in locks out the writers */
        sem_wait(&room);
    sem_post(&rlock);

    printf("reader %ld reads %d     (readers inside: %d)\n", id, value, readers);
    usleep(200000);

    sem_wait(&rlock);
    if (--readers == 0)       /* the last reader out lets the writers in */
        sem_post(&room);
    sem_post(&rlock);
    return NULL;
```

```c
}

void *writer(void *arg) {
    long id = (long)arg;
    sem_wait(&room);          /* alone in the room */
    value += 10;
    printf("writer %ld writes %d (alone)\n", id, value);
    usleep(200000);
    sem_post(&room);
    return NULL;
}

int main(void) {
    sem_init(&rlock, 0, 1);
    sem_init(&room, 0, 1);
    pthread_t r[READERS], w[WRITERS];
    pthread_create(&w[0], NULL, writer, (void *)1L);
    for (long i = 0; i < READERS; i++)
        pthread_create(&r[i], NULL, reader, (void *)(i + 1));
    pthread_create(&w[1], NULL, writer, (void *)2L);
    for (int i = 0; i < READERS; i++) pthread_join(r[i], NULL);
    for (int i = 0; i < WRITERS; i++) pthread_join(w[i], NULL);
    return 0;
}
```

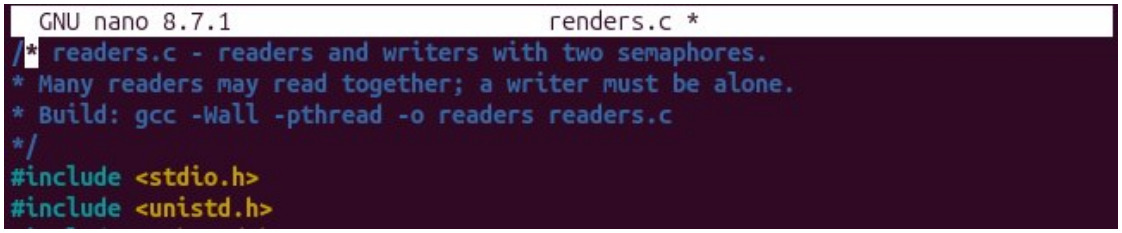


Figure 3.1: The beginning of the program in the editor

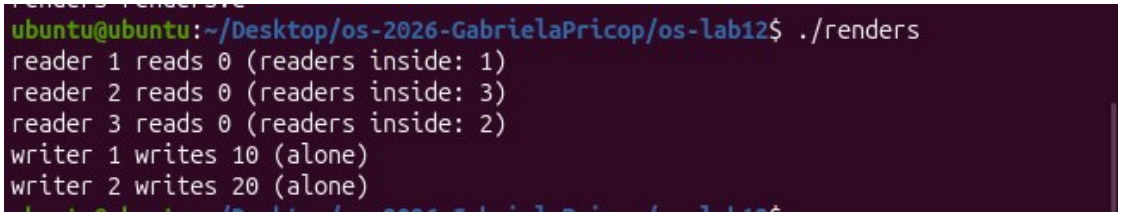


Figure 3.2: Three readers inside together, then the two writers alone

gcc -Wall -pthread -o readers readers.c ; ./readers – in this run the three readers arrived before the first writer managed to take the room, so all three read the value 0, together, and the counter printed beside them rises as they enter. After they leave, the last of them posts room, and the two writers go through one after the other, alone, printing 10 and then 20. The run is therefore a different interleaving from the one printed in the task sheet, where the first writer arrived first and the readers then read 10, but the guarantee is the same in both: the lines of the writers never overlap with anything.

A detail of the printed counter – the three readers printed the numbers 1, 3 and 2, in that order, and not 1, 2, 3. This is not an error of the synchronization, and the value of readers was never wrong: the printf is placed outside the region protected by rlock, so between the moment a reader increments the counter and the moment it reads it again to print it, another reader can increment it as well. The second reader therefore printed a counter that had already been raised by the third one. The lesson is the same as in the parking lot of the previous laboratory work, where the printf was deliberately kept inside the mutex: a value that is read outside the lock that protects it may already be stale when it is used. The run is summarized in Table 3.

Thread	**Reads/writes**	**Printed counter**	**State of the semaphore room**
reader 1	reads 0	1	taken by the first reader
reader 2	reads 0	3	still held by the group
reader 3	reads 0	2	still held by the group
writer 1	writes 10	alone	taken by the writer itself
writer 2	writes 20	alone	taken by the writer itself

Table 3: The observed run: the readers together, the writers one by one

Table 3 shows the two guarantees of the problem side by side: the three readers share the room and see the same value, while each writer holds the room alone, so no reader can ever see a half-written value.

Part 4: Build a deadlock, then remove it

Questions:

- In the stuck run, which thread holds which mutex, and which one is each thread waiting for? Draw the circle.
- Remove both sleep(1) lines from the original version and run it ten times. Does it still always hang? Why is that worse, not better?
- In one sentence: why can taking the locks in the same order never deadlock?

Answers:

- The thread one holds the mutex A and waits for B; the thread two holds the mutex B and waits for A. The circle is therefore: one holds A, one waits for B, B is held by two, two waits for A, A is held by one, and the path closes on itself. Each of the two waits for a resource that only the other can release, and neither of them will ever release anything, because both are blocked before reaching their unlock lines. Nothing is broken and nobody made an illegal call: the four actions are perfectly legitimate on their own, and only their combination is fatal. The two lines printed on the screen, one: I have A, I want B and two: I have B, I want A, are the two halves of that circle, and the program prints them before falling silent, as shown in Table 4.

- No, it does not always hang; most of the runs finish normally and print done. Without the pause, the thread one usually takes A and B one immediately after the other, before the thread two has even started, so it finishes, releases both, and then the thread two takes them in its turn without ever meeting resistance. That is worse and not better, because the defect has not been removed: it is still there, and it depends only on the timing. A bug that appears once in ten runs, or once in ten thousand, passes the tests of the programmer, passes the review, and shows itself in production, on a loaded machine, in a run that nobody can reproduce. The sleep of the original program does not create the deadlock, it only makes it reliable, which turns a dangerous bug into a visible one; this is a technique worth remembering for testing concurrent code.

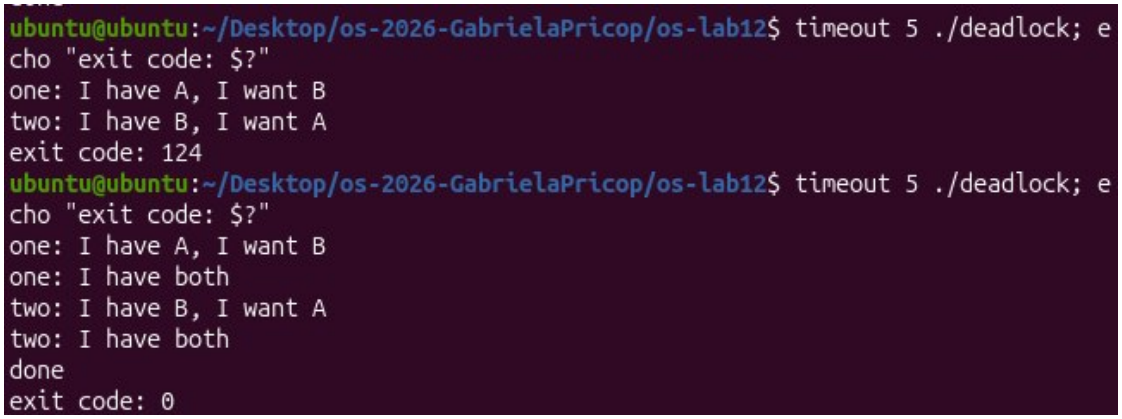


Figure 4.1: The program with the both sleep(1) removed

- Because a circle needs every thread to hold a lock that comes before the one it waits for, and if all the threads take their locks in the same global order, a thread that is waiting holds only locks that come earlier in that order, so the chain of waits always advances in one direction and can never close on itself.

deadlock.c – two mutexes and two threads, written so that each of them takes the two locks in the opposite order from the other. The sleep of one second between the two lock calls guarantees that both threads have taken their first lock before either of them asks for the second one, which makes the circle certain instead of probable. The program is run under timeout, so that the terminal comes back after five seconds even when the program never ends; the exit code 124 is how timeout reports that it had to kill the program. The source is given below.

```c
/* deadlock.c - two threads, two mutexes, taken in opposite
   order.
 * Build: gcc -Wall -pthread -o deadlock deadlock.c
 * Run:   timeout 5 ./deadlock; echo "exit code: $?"   (124 =
   killed by timeout)
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;
```

```c

void *one(void *arg) {
    pthread_mutex_lock(&A);
    printf("one: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);          /* waits for two */
    printf("one: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

void *two(void *arg) {
    pthread_mutex_lock(&B);          /* the fix: lock A first
   here too */
    printf("two: I have B, I want A\n");
    sleep(1);
    pthread_mutex_lock(&A);          /* waits for one */
    printf("two: I have both\n");
    pthread_mutex_unlock(&A);
    pthread_mutex_unlock(&B);
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, one, NULL);
    pthread_create(&t2, NULL, two, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("done\n");
    return 0;
}
```

```
 GNU nano 8.7.1                    deadlock.c *
/* deadlock.c - two threads, two mutexes, taken in opposite order.
* Build: gcc -Wall -pthread -o deadlock deadlock.c
* Run: timeout 5 ./deadlock; echo "exit code: $?" (124 = killed by timeout)
*/
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;
void *one(void *arg) {
```

Figure 4.2: The beginning of deadlock.c, with the two mutexes

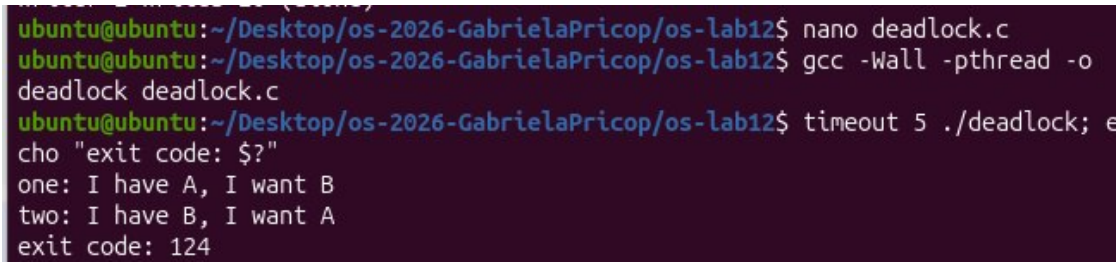


Figure 4.3: The stuck run: two lines, five seconds of silence, exit code 124

timeout 5 ./deadlock; echo "exit code: $?" – the two threads print one: I have A, I want B and two: I have B, I want A, and then the terminal stays silent. The line one: I have both never appears, nor does done, and after five seconds timeout kills the program and the shell prints exit code: 124. The program did not crash and consumed no processor time while it was stuck: both threads were asleep inside their second pthread_mutex_lock, waiting for something that would never arrive. A deadlock is a very quiet failure, which is part of what makes it dangerous.

```
printf("one: I have both\n");
pthread_mutex_unlock(&B);
pthread_mutex_unlock(&A);
return NULL;
}
void *two(void *arg) {
pthread_mutex_lock(&A); /* the fix: lock A first here too */
printf("two: I have B, I want A\n");
sleep(1);
pthread_mutex_lock(&B); /* waits for one */
printf("two: I have both\n");
pthread_mutex_unlock(&B);
pthread_mutex_unlock(&A);
```

Figure 4.4: The fixed version of deadlock.c



Figure 4.5: The run of the fixed version

The fix of deadlock.c – in the function two the two lock calls were exchanged, and the two unlock calls with them, so that both threads now take A first and B second. The thread two now waits for A while holding nothing, so no circle can form. And this is a way to solve deadlocks, to ensure the lock order is always the same, so no threads keep on waiting for each other forever.

Version	Thread one	Thread two	Result	Exit code
original	A, then B	B, then A	the circle closes, both wait for ever	124
repaired	A, then B	A, then B	one finishes, then the other	0

Table 4: The same two threads, before and after the order was imposed

Table 4 shows that the repair changed no tool and removed no lock: both mutexes are still taken, both critical sections are still protected, and the only thing that changed is the order. A deadlock is not a defect of the locks but of the order in which they are asked for.

Conclusion

The laboratory work put together the tools that the previous one had used one at a time. In the first part, two semaphores and a mutex solved the problem of the producer and the consumer: the semaphore empty, started at four, stopped the fast producer as soon as the buffer was full, and the semaphore full, started at zero, made the consumer wait at the beginning, when there was nothing to take. The runs showed the count rising to four and never to five, falling and rising again as the slow consumer freed the slots, and a second run showed a different interleaving with the same limits, which is what a correct synchronization guarantees: the order is free, the invariants are not. The third question of that part was the most instructive, because locking the mutex before the semaphore would make the program stop for ever when the buffer became full, the producer sleeping with the lock in its hand and the consumer unable to take it: a thread must never fall asleep while holding a lock.

In the second part the same problem of order as in the previous laboratory work was solved with a condition variable, and the comparison between the two solutions showed what a condition variable is not: it keeps no value and remembers nothing, so the state must live in a separate variable, here the integer ready, protected by the mutex. Replacing while by if did not change the output of this particular program, which is exactly why the experiment is worth doing: the test passed although the code had become wrong, because a thread can return from pthread_cond_wait spuriously, or find that another thread has consumed the condition while it was taking the mutex back.

In the third part many readers read together while each writer stayed alone, which two lines of the reader make possible: the first reader to arrive takes the semaphore room and the last one to leave gives it back. The counter of the readers needed a lock of its own, because without it two readers could both believe they were the first, or two could both release the room. The observed run also showed a detail that is easy to miss: the counters printed by the three readers were 1, 3 and 2, because the printf is outside the lock that protects the counter, so the value printed may already be out of date even though it was never wrong. A continuous stream of readers would make the second writer starve, since the counter would never fall to zero.

In the fourth part a deadlock was built on purpose from two mutexes taken in opposite order, and the run under timeout showed its quiet signature: two lines, five seconds of complete silence and the exit code 124. The circle is that the thread one holds A and waits for B while the thread two holds B and waits for A. Removing the pauses would make the program finish most of the time, which is worse rather than better, because the defect would become intermittent instead of disappearing. Taking both locks in the same order removed it for good, and the exit code became 0: a chain of waits that always advances in the same direction can never close on itself.

References

1. *Gabriela Pricop. os-2026-GabrielaPricop: laboratory works of the Operating Systems course, GitHub repository*

https://github.com/gabriela-pricop/os-2026-GabrielaPricop

2. *pthread_cond_wait(3) – wait on a condition variable, and why the condition must be re-tested, Linux manual page*
   https://man7.org/linux/man-pages/man3/pthread_cond_wait.3.html

3. *sem_overview(7) – overview of POSIX semaphores, Linux manual page*
   https://man7.org/linux/man-pages/man7/sem_overview.7.html

4. *Allen B. Downey, The Little Book of Semaphores: the producer-consumer and the readers-writers problems*
   https://greenteapress.com/wp/semaphores/